reduces the risk of leaks or errors, while making it easier to add or update tools without touching the AI itself.

In short, using the MCP with Next.js provides a modular, flexible, and safer integration model for building web applications enhanced by LLMs.

12.4 Inside an MCP server: Extending web applications

Before looking at a more complex integration of MCP servers with LangChain.js, we need to take a closer look at the main components of an MCP server. We've now seen MCP servers in action, but to understand their full potential, we need to look at their core building blocks. An MCP server exposes tools, prompts, and resources to clients or agents. These components form the interface between external systems and the AI model, making the system both extensible and modular.

Figure 12.5 shows the main building blocks of an MCP system and how they relate to applications, clients, servers, and external services. Any application, whether a web UI, CLI tool, or script, acts as a consumer, sending requests to a local or colocated MCP client (also called an agent or SDK). The MCP client discovers and invokes tools, prompts, and resources exposed by an MCP server, which may be local or remote. The MCP server streams responses back through the client to the application, enabling incremental updates and tool integration. Figure 12.5 emphasizes the roles and flows rather than specific deployment locations.

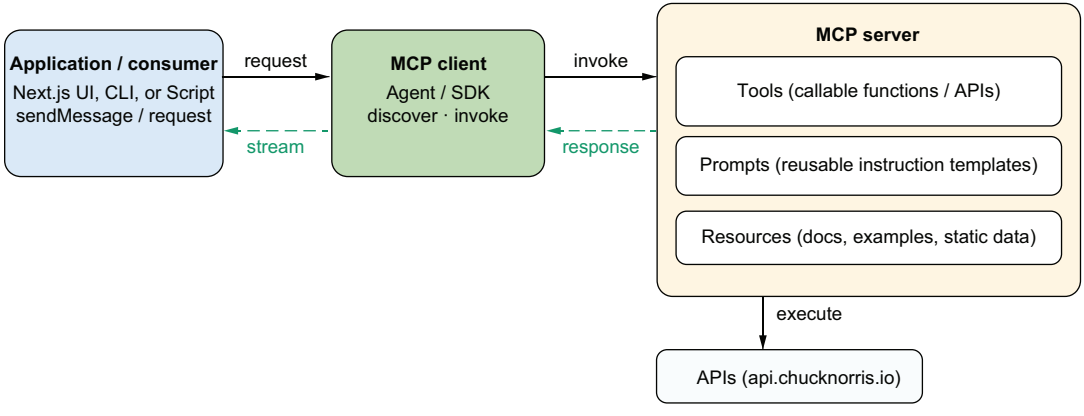


Figure 12.5 MCP server building blocks and how they interact with clients and external services

NOTE The MCP Inspector is a graphical tool that comes with the local MCP server. It allows you to explore the server's available integrations, including tools, prompts, and resources, without needing to go to the MCP website. You can run it locally alongside your MCP server to inspect, debug, and experiment with the system in real time.